

Nyasha Hama

Software Engineer

Full-stack products · React, TypeScript, Go & PostgreSQL

Cape Town, South Africa | nyashahama5@gmail.com | +27 69 785 2182 | nyashahama.xyz | GitHub | LinkedIn

PROFILE

Full-stack engineer building web products with React, Next.js, Go and PostgreSQL. Own work from user workflows and APIs to data modelling and automated tests. Built offline reporting and payment reconciliation systems; contributed database correctness fixes to Turso and Python/C tracing improvements to CrossHair.

TECHNICAL SKILLS

Frontend: TypeScript, React, Next.js, Tailwind CSS, responsive interfaces

Backend & Data: Go, SQL, PostgreSQL, REST APIs, pgx, sqlc, Redis; also Rust and C#

Delivery & Testing: Docker, GitHub Actions, Playwright, Vitest, Linux, structured logging

EXPERIENCE

Independent Software Engineer

Jan 2023 – present

Self-directed product development across frontend, backend and data, including the projects below. Own implementation, regression tests and automated delivery pipelines.

Full-Stack Development | Confidential Project

Contribute to an ongoing full-stack software project, including hands-on C# development. Client and implementation details are confidential under NDA.

SELECTED PROJECTS

ClinicPulse | [Source](#) | [Demo](#)

Clinic operations | Alpha demo | Next.js, TypeScript, Go, PostgreSQL

- Built field-reporting, district-review and admin workflows with role-based access, connecting Next.js interfaces to Go APIs and PostgreSQL.
- Preserved field reports across connectivity loss with a persisted browser queue. Handled duplicate and conflicting submissions explicitly, with regression tests for concurrent retries and timestamp precision.

StrataHQ | [Source](#) | [Demo](#)

Property operations | Seeded beta demo | Next.js, TypeScript, Go, PostgreSQL

- Built managing-agent, trustee and resident workflows for levies, maintenance and governance, with scheme-scoped access to property records.
- Implemented bank-statement reconciliation with stable row fingerprints and transactional payment updates. Routed ambiguous matches and overpayments to manual review; tested matching, partial payments and import identity.

TxProof | [Source](#)

Bounded payment failure testing | Rust, PostgreSQL

- Built a test harness for payment retries, webhooks and restarts, with deterministic replay and PostgreSQL invariant checks. Reproduced duplicate provider objects after a lost response and verified the corrected retry behaviour in a synthetic reference application.

OPEN SOURCE CONTRIBUTIONS

Turso | Rust | [PR #6993](#) / [#7117](#)

May 2026

- Fixed AUTOINCREMENT metadata loss after column removal and stale sequence state after column changes in Turso's SQLite-compatible engine. Added regression coverage; changes merged upstream.

CrossHair | Python/C | [PR #413](#)

May 2026

- Moved call-target normalization into CrossHair's C tracer while preserving callable dispatch and error behaviour. Added regressions for bound methods and callable objects; change merged upstream.

EDUCATION

University of Johannesburg

2022-2024

Coursework toward BSc Computer Science, completed through final year.